

M3.11 Team Deliverable 3: Priority Plan & Tech Exploration

Smart Factory Team

Team Members: Hoang Nguyen, William Miller, Chase Post, Shiloh Parker, Thierry Tran

Instructions:

In this module, your goal is to create and summarize two items, and create a sprint backlog for sprint 1.

1. The Priority Plan will be the result of Planning Poker and MoSCoW. This will require enumerating goals and build components, then conducting your own version of Planning Poker from Module 3. The goal is to come to consensus about what might be possible for your project. At this point, there is still time to reduce scope. After or during the Planning Poker stage, you will also need to establish a MoSCoW prioritization (i.e., Must-Should-Could-Won't). While you are free to develop your own scheme for assigning priority and estimating difficulty, the overall process should be described.

The Priority Plan should be structured as follows:

- Planning Poker approach—How planning poker was done by your team
 - Key goals and tasks—The most important priorities of goals and build components/features
 - Considered goals and features—An enumeration of all considered goals and build components/features
2. The Tech Exploration report will include a summary of small tests of the development stack your team is considering. For example, if your team is expecting to use Docker, at least one person on your team should download it and test it. Another example would be testing key aspects of software if you are already aware that it is crucial for your development. If you are building a game, for example, common platforms include Unity and Unreal Engine. Having people test both can help your team make decisions about which technologies should be applied.

The Tech Exploration report should be structured as follows:

- Considered technology and reasoning—A list of considered technology (such as APIs, libraries, or platforms)
 - Initial tech testing—A summary of what was done in terms of testing technology
 - Tech exploration conclusions—A short statement about whether considered technology has led to a decision and what it was. For example, Unity and why it was chosen.
3. Sprint Backlog. The sprint backlog is a list of items to be accomplished in this 2 week sprint. Each backlog item should include a title, description, and a person assigned to accomplish the task. Each person should be assigned at least 3 items. You may use choose the format of your sprint backlog. You may use github issues or a github

taskboard, tools like Jira or Trello, a spreadsheet, or any other tool of your group's choosing to maintain your sprint backlog. The backlog items will come from your Priority Plan and the Tech Exploration Report.

To receive credit for your progress, please attach the following documentation.

1. Your Priority Plan PDF
2. Your Tech Exploration report PDF
3. Scrum meeting minutes PDF
4. A link to your team's sprint backlog or a PDF document containing the sprint backlog
5. A link to your teams product backlog or a PDF document containing the product backlog

You should get the sprint backlog finished as soon as possible because you need to be working on the backlog items right away. We will have a sprint review meeting, and you will need to have some features working and ready to demonstrate. You need to start working on those features right away. Do not wait until the last minute!

Priority Plan

I. Planning Poker Approach

- Participants' wants/needs/preferences will be rated by each member in the team, in terms of how much effort it will take to do.
- If there are any differences between ratings, members will discuss and finalize the rating.
- We used a simple Planning Poker approach to estimate effort for each major feature. Each team member independently gave a rough effort score using a small Fibonacci-style scale (1, 2, 3, 5, 8). After revealing our numbers, we discussed any large differences in estimates to understand assumptions or misunderstandings about scope. If needed, we re-voted after discussion. This helped us reach a shared understanding of how complex each feature would likely be and where we might need to reduce scope.

II. Key goals and tasks

- Have a configurable, intuitive user interface.
- Have a database that stores relevant parameters, variables, and outputs securely.
- Able to perform well under high data load, so end users are not slowed down by the application's performance.
- For Sprint 1, we agreed to focus on building one complete piece of working functionality rather than spreading ourselves too thin. Specifically, we will focus on the smelting process only. That means building parameter input, backend simulation logic, storing results in the database, and displaying outputs in the frontend. The other processes (scalping, milling, heat treating, shipment) will be addressed in later sprints once we have a stable foundation.

III. Considered goals and features

MUST HAVE:

- Ability to model and optimize an aluminum production process
- Model the smelting/casting
- Model the scalping process
- Model the milling process
- Model the heat-treating process
- Model the product shipment process
- Relies on a database that stores relevant parameters, variables, and outputs securely
- Ability to run simulations and see reports
- Ability to easily view, modify, and maintain the code for this application
- Ability to set parameter values for different stages of the process
- Have a configurable, intuitive user interface
- Able to perform well under high data load
- Able to see the outcome of a simulation in clear terms

SHOULD HAVE:

- Be compliant with at least 1 cybersecurity framework to mitigate cyber risk

- Be set up using Python in the backend and React in the Frontend, with a standard framework connecting the two
- Ability to connect the cloud-based production model to real-time, real-world data feeds,
- Able to see control charts corresponding to a simulated operation

COULD HAVE:

- Have an authentication panel for various types of users

WON'T HAVE:

- Have a distinct UI/UX for various types of users

Tech Exploration

I. Considered technology and reasoning

- We discussed several technology options before deciding on our stack. Python was the only consideration for our backend. For the communication between the frontend and backend, we considered Node.js, FastAPI, Flask, and Django. For the frontend, we discussed React and Next.js. For the database, we considered PostgreSQL, SQLite, and MySQL. Our main criteria were ease of setup, team familiarity, integration complexity, and how quickly we could get a working demo. Since this is a short sprint, we leaned toward tools that allow fast iteration without too much configuration overhead.

II. Initial tech testing

- We set up a local SQLite database and created a basic schema for the smelting process. A simple Python script was written to test database connectivity and insert sample records. On the frontend side, we tested basic React setup and verified that we could structure components for parameter input and results display. For the API, FastAPI was experimented with to understand its file structure and Python integration, as no one from our group has had experience with it; a simple server was set up on a local machine.

III. Tech exploration conclusions

- Based on our testing and discussion, we decided to proceed with Python for the backend and React for the frontend. We will use a local SQLite database during Sprint 1 to minimize setup complexity, as our database needs are not grand. If time allows in later sprints, we may consider moving to a cloud-hosted database or expanding infrastructure features. For the API, Django was most highly considered, but a positive experience testing FastAPI swayed our opinion. Django would be too much overhead for our short sprint. We have proceeded with FastAPI rather than Node.js because FastAPI is natively in Python, allowing potentially mathematically-heavy workflows to be coded without extra microservices. We may incorporate a mix of REST and WebSockets, if needed.

Sprint Backlog

Our goal for this sprint is to deliver a basic but fully connected version of the smelting process. By the end of the sprint, we want to demonstrate working software where a user can input parameters, run a simulation, store results in the database, and view outputs on the dashboard.

As a production analyst, I'd like the ability to model and optimize an aluminum production process, so I can run a line that maximizes profit.

- As a production analyst, I'd like to model pure aluminum block/can inventory/storage so I can optimize parameters and operations.
- As a production analyst, I'd like to model the smelting/casting so I can optimize parameters and operations.
 - Backend logic initial framework for smelting. Assigned (2/19): Hoang
 - Alloy selection functionality, each with different weights and processing times. Assigned (2/19): Hoang
 - Build ability to simulate operation for a certain amount of time and store results. Assigned (2/19): Hoang
 - Build parameter selection UI for smelting to integrate with backend and allow user input, working with Hoang for integration. Assigned (2/19): Chase
 - Build basic visualization for smelting operation in React. Assigned (2/19): Chase
 - Build charting or dashboard to show system performance. Assigned (2/19): Chase
 - Build control panel to define and run simulations. Assigned (2/19): Thierry
- As a production analyst, I'd like to model the scalping process so I can optimize parameters and operations.
- As a production analyst, I'd like to model the milling process so I can optimize parameters and operations.
- As a production analyst, I'd like to model the heat treating process so I can optimize parameters and operations.
- As a production analyst, I'd like to model the product shipment process so I can optimize parameters and operations.

As an IT engineer, I'd like the process to rely on a database that stores relevant, parameters, variables, and outputs securely.

- Create database in AWS with basic configuration. Assigned (2/19): William

- Develop database schema to support smelting process in collaboration with Hoang. Assigned (2/19): William
- Populate database tables with useful sample data to run testing of smelting process. Assigned (2/19): William
- Connect database to backend for smelting process modeling, working with Hoang and Chase. Assigned (2/19): Shiloh

As an IT engineer, I'd like the application to be setup using Python in the backend and React in the Frontend with a standard framework connecting the two.

- Selecting best framework and method for React-Python integration. Assigned (2/19): Shiloh
- Building high-level file structure to support the end-to-end aluminum coil process. Assigned (2/19): Shiloh
- Defining necessary API logic for front-end for smelting, working with Shiloh and Chase. Assigned (2/19): Thierry
- Defining necessary API logic for back-end for smelting, working with Shiloh and Hoang. Assigned (2/19): Thierry

As a production analyst, I'd like the ability to run simulations and see reports to understand the dynamics of our aluminum production process.

As an IT engineer, I'd like the ability to easily view, modify, and maintain the code for this application, so I can keep my end-users satisfied.

As a security analyst, I'd like the application to be compliant with at least 1 cybersecurity framework, so I can mitigate cyber risk.

As a production controller, I'd like the ability to connect the cloud-based production model to real-time, real-world data feeds, so I can have a digital twin that supports my decision-making.

As a controls engineer, I'd like the ability to set parameter values for different stages of the process, so my simulations translate into real decisions.

As a controls engineer, I'd like to see the outcome of a simulation in clear terms, so I can more accurately estimate the impact of a change on the process's performance.

As a controls engineer, I'd like to see control charts corresponding with a simulated operation, so I can better estimate the impact of a change on the frequency of defects like coil thickness and tension issues.

As a project manager, I'd like the application to have a configurable, intuitive user interface, so end users do not have issues adopting it in their daily work.

As a project manager, I'd like the application to perform well under high data load, so end users are not slowed down by the application's performance.